

Neural Networks, Perceptron, Backprop & Chain Rule, PyTorch Basics

Sattam Altwaim

sattam.tuuaim@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

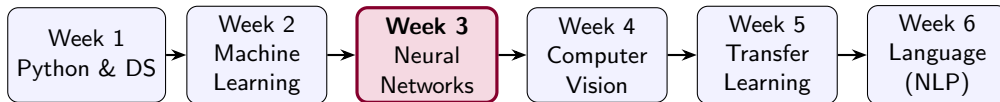


موهبة
Mawhiba

KAUST Academy & Mawhiba
AI Olympiad Training (Beginners)
Week 3, Day 15, Lecture

3 October 2026

1. Big Picture and Motivation
2. Learning Outcomes
3. The Neuron
4. Activation Functions
5. The Network
6. Backpropagation and the Chain Rule
7. PyTorch Basics
8. Olympiad Practice
9. Limitations and What Comes Next
10. Summary
11. References



Today: Day 15 of 42

- ▶ **Week 2:** a model is a function with dials, chosen by gradient descent (Day 08); judged on held-out rows (Day 09); lines, neighbours, trees and forests (Day 10). Every bend in those models was typed by us: x^2 , a midpoint, a log.
- ▶ **Today:** the model that **invents its own columns**: the neural network. Its parts are Day 08's logistic regression; its training is Day 08's loop plus one rule, the chain rule; its tool is PyTorch.
- ▶ **This week:** tomorrow, loss functions, mini-batches and the full training loop; Monday, the dials that make training reliable; then three self-study days and the Friday competition.

How We Did It Before: Columns Typed by Hand

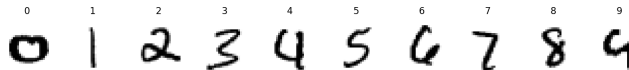


Source: 1,797 handwritten digits of 8×8 pixels (Alpaydin and Kaynak, 1998), `sklearn.datasets.load_digits`; one example per class; `scripts/day15_nn_figures.py`.

The Week 2 recipe on these digits: a person chooses the columns, then a line (one logistic regression per digit) learns its dials.

Columns given to the line	Who chose them	Validation accuracy
16 hand-made: the sum of each row and of each column	us	0.881
the 64 raw pixels	nobody, the image as it is	0.972

Our 16 columns threw away what the line needed; on 8×8 pixels the raw image plus a line is already strong (1,437 training images, 360 for validation).



Source: MNIST (LeCun et al., 1998), 28×28 pixels, via OpenML `mnist_784`; one test image per class; `scripts/day15.nn_figures.py`.

The problem

- ▶ Make the image 28×28 (784 pixels) and one line per digit scores **0.892** on 10,000 test images: 1,080 digits wrong. A line adds up pixels; it cannot say “a stroke here *and* a gap there”.
- ▶ Type the missing columns? “A stroke at the top, then a diagonal” is a 7, until it is slanted, thin or a child’s. Nobody can write the 7-detector by hand.
- ▶ Every bend we ever had was typed by us and tamed by hand: Day 09’s degree 3, Day 10’s midpoint and log. The model never invented a column.

The pattern

What a person cannot write, the data can choose: the columns themselves must get dials, and gradient descent must turn them.

In one sentence

A **neural network** is Day 08's logistic regression used as a building block: a layer of them computes new columns from the inputs, the next layer draws its line through those columns, and gradient descent turns every dial at once.

- ▶ A **neuron** is a line plus a bend (an activation function). Without the bend, stacking adds nothing; with it, two neurons already solve what one line cannot.
- ▶ The loss depends on the first layer only *through* the second. The **chain rule** multiplies the slopes along that chain; applied layer by layer it is **backpropagation**, and `loss.backward()` does it for you.
- ▶ **PyTorch** gives tensors (NumPy arrays with slopes), ready-made layers, and the same four-move training loop as Day 08.

Where you will meet it

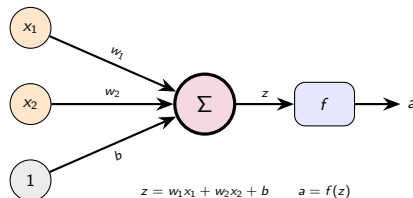
Every IOAI task on images, sound or text starts from a network, and the models of Weeks 4 to 6 are this one with better-shaped layers. On a table, today's network is one more model to compare against the line and the forest, by the task's metric, on held-out rows.

By the end of this session, you will be able to:

- ▶ **Explain** what one neuron computes, and why it is Day 08's logistic regression with a new name.
- ▶ **Say** why a network needs activation functions, and place ReLU, sigmoid and softmax where they belong.
- ▶ **Compute** the forward pass of a small network by hand, and count its parameters.
- ▶ **Apply** the chain rule to get the gradient of one neuron, and explain how backpropagation carries it through a hidden layer.
- ▶ **Build** a network in PyTorch from tensors, `nn.Linear` and `nn.Sequential`, and train it with the four-move loop.
- ▶ **Recognise** a task where a network beats a line, and one where it does not.

Neural Networks: **The Neuron**

Definition: A Neuron Is Logistic Regression With a New Name



Definition

A **neuron** takes the inputs $\mathbf{x}$, computes the line $z = \mathbf{w} \cdot \mathbf{x} + b$ (Day 08), and passes it through an **activation function** f : the output is $a = f(z)$.

- $\mathbf{w}$, b : the dials, one weight per input.
With $f = \sigma$ the neuron is Day 08's logistic regression; with $f = \text{ReLU}$ it is the usual unit inside a network.

The circle adds up, the box bends; every model of this week is made of this picture.

The versicolor flower of row 50 (Day 08): petal length $x_1 = 4.7$, petal width $x_2 = 1.4$. Dials $\mathbf{w} = (1, -2)$, $b = -1$.

$$z = 1 \cdot 4.7 + (-2) \cdot 1.4 + (-1) = 4.7 - 2.8 - 1 = 0.9$$

$$a = \sigma(0.9) = \frac{1}{1 + e^{-0.9}} = \frac{1}{1 + 0.407} = 0.711 \quad (\text{sigmoid: a probability})$$

$$a = \text{ReLU}(0.9) = \max(0, 0.9) = 0.9 \quad (\text{ReLU: the number itself, negatives become 0})$$

With the sigmoid this neuron says “71% yes”. With ReLU it says “0.9”: not a probability, a **new column** computed from the two old ones. Training (later today) chooses the dials.

Check yourself

Same flower, dials $\mathbf{w} = (-1, 3)$, $b = 1$: $z = -4.7 + 4.2 + 1 = 0.5$, $\text{ReLU}(0.5) = 0.5$. Keep both numbers, 0.9 and 0.5: the network section uses them.

Definition

The **perceptron** (Rosenblatt, 1958) is a neuron whose activation is the **step**: $a = 1$ if $z \geq 0$, else 0. Yes or no, never “71%”.

- ▶ It learns from its mistakes, one row at a time: when wrong, $\mathbf{w} \leftarrow \mathbf{w} + \alpha (y - a) \mathbf{x}$ and $b \leftarrow b + \alpha (y - a)$; when right, nothing moves.
- ▶ If a line can separate the classes, the rule finds one after finitely many mistakes.

One mistake, with numbers

$\mathbf{w} = (0, 0)$, $b = 0$, $\alpha = 1$. Row (4.7, 1.4), $y = 1$: $z = 0$, $a = 1$, right, no change.

Row (1.4, 0.2), $y = 0$: $a = 1$, wrong:
 $\mathbf{w} \leftarrow (0, 0) + (0 - 1)(1.4, 0.2) = (-1.4, -0.2)$, $b \leftarrow -1$.

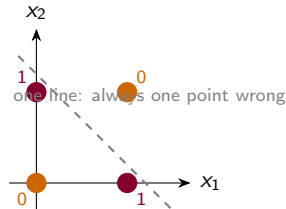
Why we no longer use the step

The step has no slope: a small change of $\mathbf{w}$ changes nothing, then everything. Gradient descent needs a slope: sigmoid or ReLU instead.

Whatever the activation, the answer flips where $z = \mathbf{w} \cdot \mathbf{x} + b = 0$: a straight line (Day 08's decision boundary).

x_1	x_2	y (XOR: exactly one is 1)
0	0	0
0	1	1
1	0	1
1	1	0

No line puts the two 1s on one side and the two 0s on the other (Minsky and Papert, 1969).



The way out

Two neurons draw two lines; a third combines their answers. That network (next sections) draws XOR exactly.

Neural Networks: **Activation Functions**

Feed one line into another: $z = 2x + 1$, then $y = 3z - 4$.

$$y = 3(2x + 1) - 4 = 6x - 1 \quad \text{check at } x = 2 : 3 \cdot (2 \cdot 2 + 1) - 4 = 11 = 6 \cdot 2 - 1$$

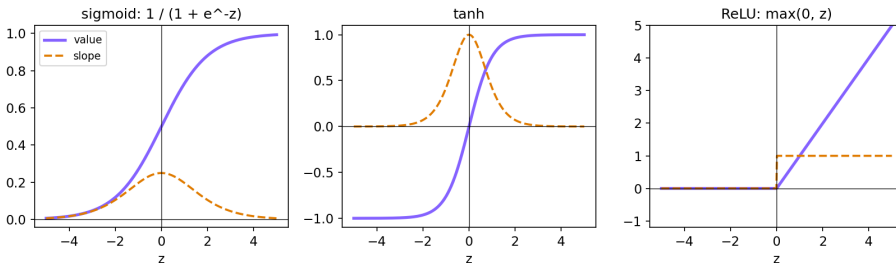
- ▶ Two lines in a row are one line (slope 6, intercept -1). The same holds with matrices:
 $\mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (\mathbf{W}_2\mathbf{W}_1)\mathbf{x} + (\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2)$.
- ▶ A hundred linear layers can still only draw what Day 08's one line draws.

The problem

Stacking adds nothing unless something **bends** between the layers. That something is the activation function.

Definition

An **activation function** is a fixed curve f applied to a neuron's z , one number in, one number out. It has no dials; its only job is to bend.



Source: the three functions and their slopes evaluated on 400 points; `scripts/day15_nn_figures.py`.

The dashed slope is what gradient descent will use. Where it is 0, the dials stop moving.

Name	Formula	$f(-2)$	$f(0)$	$f(2)$	Slope
sigmoid σ	$1/(1 + e^{-z})$	0.119	0.5	0.881	$\sigma(z)(1 - \sigma(z))$, at most 0.25
tanh	$(e^z - e^{-z})/(e^z + e^{-z})$	-0.964	0	0.964	$1 - \tanh^2(z)$, at most 1
ReLU	$\max(0, z)$	0	0	2	1 if $z > 0$, else 0

- **Sigmoid** squashes into $(0, 1)$: a probability. Far from 0 its slope is almost 0 (at $z = 5$: 0.0066): the dials freeze. People call this a **vanishing gradient**.
- **tanh** is a sigmoid centred at 0, into $(-1, 1)$. Same freezing far from 0.
- **ReLU** keeps positive numbers and kills negative ones. Slope exactly 1 on the right: nothing shrinks, and it costs one comparison.

Rule of thumb

ReLU inside the network. The output layer gets the activation the *task* needs (next frame).

Place	Task	Activation	Why
hidden layers	any	ReLU	bends, cheap, slope 1
output	regression (a number)	none	any value allowed, like Day 08's line
output	yes / no	sigmoid	one probability, like logistic regression
output	one of k classes	softmax	k probabilities that add to 1

- ▶ The output activation is the same decision as Day 08's "is the target a number or a category?" .
- ▶ In PyTorch the softmax and the sigmoid of the output are usually folded into the loss function (CrossEntropyLoss, BCEWithLogitsLoss); the network's last layer then outputs raw scores called **logits**. Day 16 explains why.

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}$$

$\mathbf{z}$: the k raw scores of the output layer (one per class); e^{z_i} makes every score positive; dividing by the sum makes them add to 1.

Three classes

Scores $\mathbf{z} = (2.0, 1.0, 0.5)$: $e^{2.0} = 7.389$, $e^{1.0} = 2.718$, $e^{0.5} = 1.649$, sum 11.756.

Probabilities $(7.389, 2.718, 1.649)/11.756 = (0.629, 0.231, 0.140)$, which add to 1.

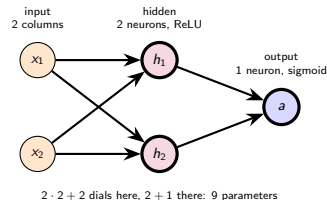
The prediction is class 1, with 63%.

- ▶ The biggest score always gets the biggest probability, so argmax of the scores is already the predicted class.
- ▶ With $k = 2$ the softmax is the sigmoid of the difference of the two scores: nothing new for yes/no.

Neural Networks: **The Network: Layers of Neurons**

Definition

A **layer** is a group of neurons reading the same inputs, each with its own dials. A **neural network** (multi-layer perceptron, MLP) stacks layers: one layer's outputs are the next one's inputs.



- **Input:** the table's columns, no dials.
Hidden: the invented columns; its size is a dial. **Output:** 1 neuron for a number or yes/no, k for k classes.
- **Fully connected:** every neuron reads every input; each arrow is a weight, each neuron adds a bias.

The 2-2-1 network above on the flower $\mathbf{x} = (4.7, 1.4)$, with the dials of the neuron section:

$$\text{hidden, neuron 1: } z_1 = 1 \cdot 4.7 + (-2) \cdot 1.4 - 1 = 0.9, \quad h_1 = \text{ReLU}(0.9) = 0.9$$

$$\text{hidden, neuron 2: } z_2 = (-1) \cdot 4.7 + 3 \cdot 1.4 + 1 = 0.5, \quad h_2 = \text{ReLU}(0.5) = 0.5$$

$$\text{output: } z = 2 \cdot 0.9 + (-4) \cdot 0.5 + 0 = -0.2, \quad a = \sigma(-0.2) = 0.450$$

- ▶ The output neuron never saw the flower. It saw $(0.9, 0.5)$, two columns the hidden layer computed. Change the hidden dials and the output neuron works with different columns.
- ▶ $a = 0.450 < 0.5$: the network says “no”. The flower is a versicolor ($y = 0$): right, with loss $(0.450 - 0)^2 = 0.203$ to shrink.

Why this beats one line

The two hidden neurons are two lines through the flowers; the output neuron combines where the point falls relative to both. That is exactly the two-line fix for XOR, learned instead of typed.

For a whole table $\mathbf{X}$ (n rows, d columns), a hidden layer of m neurons and k outputs:

$$\mathbf{H} = \text{ReLU}(\mathbf{X}\mathbf{W}_1^\top + \mathbf{b}_1), \quad \mathbf{Z} = \mathbf{H}\mathbf{W}_2^\top + \mathbf{b}_2$$

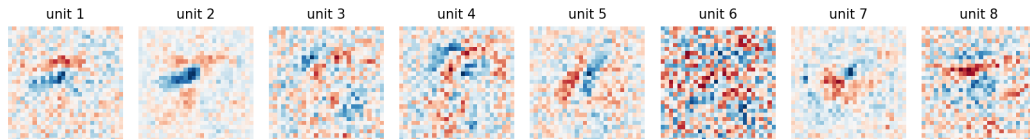
- ▶ $\mathbf{W}_1$: $m \times d$, one row of weights per hidden neuron; $\mathbf{b}_1$: m biases. $\mathbf{W}_2$: $k \times m$; $\mathbf{b}_2$: k . Day 05's $\mathbf{X} @ \mathbf{W} + \mathbf{b}$, twice, with a bend in between.
- ▶ Shapes: $\mathbf{X}$ is $n \times d$, $\mathbf{H}$ is $n \times m$ (the invented columns), $\mathbf{Z}$ is $n \times k$ (the scores).
- ▶ **Parameter count:** $d m + m + m k + k$.

Worked example

Digits, 8×8 : $d = 64$, $m = 32$, $k = 10$: $64 \cdot 32 + 32 + 32 \cdot 10 + 10 = 2048 + 32 + 320 + 10 = 2,410$ dials.

MNIST, 28×28 : $d = 784$, $m = 128$: $784 \cdot 128 + 128 + 128 \cdot 10 + 10 = 101,770$ dials.
Day 08's line had 2.

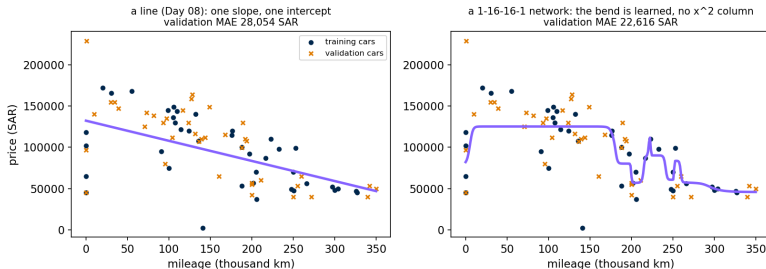
What the Hidden Layer Learns



Source: the first 8 of the 128 hidden neurons of the MNIST network trained later in this deck; each neuron's 784 weights drawn as a 28×28 image, red positive, blue negative; `scripts/day15_nn_figures.py`.

- ▶ A hidden neuron's weights have the shape of the input, so they can be looked at. Each one is a **template**: it fires when the ink lies on its red pixels and stays off its blue ones.
- ▶ Nobody drew these. They are the 128 columns the network invented so that its output layer, one line per digit, can separate the classes.
- ▶ Day 08's "deep learning finds its own columns" is this picture. Week 4's convolutions make the templates small and reusable.

The Network Bends by Itself

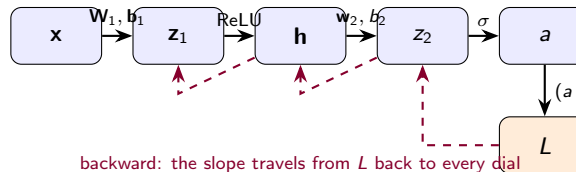


Source: the 80 Chevrolet Tahoe listings of Day 09 (Kaggle turkibintalib/saudi-arabia-used-cars-dataset), same 40/40 split; MLPRegressor with two hidden layers of 16 tanh units on the scaled mileage; `scripts/day15.nn.figures.py`.

- ▶ Day 09 bent this line by typing x^2 and x^3 columns and choosing degree 3 on validation (MAE 23,171). The network got **22,616** from the raw mileage: it chose the bend itself.
- ▶ Same data, same split, same metric. The difference is who invented the columns.

Neural Networks: **Backpropagation and the Chain Rule**

Training a Network Is the Same Walk Downhill



Day 08's four moves do not change:

1. **predict:** the forward pass, $x \rightarrow h \rightarrow a$;
2. **loss:** how wrong, for example $L = (a - y)^2$;
3. **gradient:** the slope of L for every dial;
4. **update:** $w \leftarrow w - \alpha \partial L / \partial w$.

The only new question

L depends on w_1 only through h , then z_2 , then a . How do you get the slope of a chain of functions?

Forward: numbers flow left to right. Backward: slopes flow right to left. The second flow is **backpropagation**.

The chain rule

If a depends on z , and z depends on x , then the slope of a with respect to x is the product of the two slopes:

$$\frac{da}{dx} = \frac{da}{dz} \cdot \frac{dz}{dx}$$

Worked example

$z = 3x + 1$ and $a = z^2$. At $x = 1$: $z = 4$, $a = 16$.

$\frac{dz}{dx} = 3$ (the line's slope), $\frac{da}{dz} = 2z = 8$, so $\frac{da}{dx} = 8 \cdot 3 = 24$.

Check without any rule: $x = 1.001$ gives $z = 4.003$, $a = 16.024$; the rise 0.024 over the run 0.001 is 24.0.

Analogy

Gears. Turning x once turns z three times; turning z once turns a eight times. So turning x once turns a $3 \cdot 8 = 24$ times. A chain of three gears multiplies three ratios.

One neuron, squared error: $z = w x + b$, $a = \sigma(z)$, $L = (a - y)^2$. Three links, three slopes, one product:

$$\frac{\partial L}{\partial w} = \underbrace{\frac{\partial L}{\partial a}}_{2(a-y)} \cdot \underbrace{\frac{\partial a}{\partial z}}_{a(1-a)} \cdot \underbrace{\frac{\partial z}{\partial w}}_x \qquad \frac{\partial L}{\partial b} = 2(a - y) \cdot a(1 - a) \cdot 1$$

Worked example

$w = 2$, $b = -1$, $x = 1.5$, truth $y = 1$: $z = 2 \cdot 1.5 - 1 = 2$, $a = \sigma(2) = 0.881$, $L = (0.881 - 1)^2 = 0.014$.

$\partial L / \partial a = 2(0.881 - 1) = -0.238$, $\partial a / \partial z = 0.881 \cdot 0.119 = 0.105$, $\partial z / \partial w = 1.5$.
 $\partial L / \partial w = -0.238 \cdot 0.105 \cdot 1.5 = -0.0376$ and $\partial L / \partial b = -0.0250$: both negative, so the update *raises* w and b , pushing a towards 1. Correct direction.

Day 08's gradient $\frac{1}{n} \sum 2 e_i x_i$ was this chain with no activation (the middle slope equal to 1). That is where the “trust the formula” of Day 08 came from.

The 2-2-1 network of the forward-pass frame on $\mathbf{x} = (4.7, 1.4)$, truth $y = 0$: $\mathbf{z}_1 = (0.9, 0.5)$, $\mathbf{h} = (0.9, 0.5)$, $z_2 = -0.2$, $a = 0.450$, $L = 0.203$.

$$\delta_2 = \frac{\partial L}{\partial z_2} = 2(a - y) a(1 - a) = 0.900 \cdot 0.2475 = 0.223$$

$$\frac{\partial L}{\partial \mathbf{w}_2} = \delta_2 \mathbf{h} = (0.201, 0.111), \quad \frac{\partial L}{\partial b_2} = \delta_2 = 0.223$$

$$\delta_1 = \delta_2 \mathbf{w}_2 \odot \text{ReLU}'(\mathbf{z}_1) = 0.223 \cdot (2, -4) \odot (1, 1) = (0.446, -0.891)$$

$$\frac{\partial L}{\partial \mathbf{w}_1} = \delta_1 \otimes \mathbf{x} = \begin{pmatrix} 0.446 \cdot 4.7 & 0.446 \cdot 1.4 \\ -0.891 \cdot 4.7 & -0.891 \cdot 1.4 \end{pmatrix} = \begin{pmatrix} 2.095 & 0.624 \\ -4.190 & -1.248 \end{pmatrix}, \quad \frac{\partial L}{\partial \mathbf{b}_1} = \delta_1$$

- ▶ Lines: the output's error signal; output dials = signal $\times$ their inputs; the signal passed back through $\mathbf{w}_2$ ($\odot$: entry by entry, $\text{ReLU}' = 1$ as both $z_1 > 0$); hidden dials = signal $\times$ inputs ($\otimes$: every δ times every x).
- ▶ **A dial's slope = the signal at its neuron $\times$ the input it multiplies**, layer by layer backwards: backpropagation (Rumelhart, Hinton and Williams, 1986).

```
import torch
W1 = torch.tensor([[1., -2.], [-1., 3.]], requires_grad=True)
b1 = torch.tensor([-1., 1.], requires_grad=True)
w2 = torch.tensor([2., -4.], requires_grad=True)
b2 = torch.tensor(0., requires_grad=True)
x, y = torch.tensor([4.7, 1.4]), 0.0

h = torch.relu(W1 @ x + b1) # forward pass, recorded
loss = (torch.sigmoid(w2 @ h + b2) - y) ** 2 # 0.2026
loss.backward() # the chain rule, backwards
print(w2.grad) # tensor([0.2006, 0.1114])
print(W1.grad) # tensor([[ 2.0948,  0.6240], [-4.1895, -1.2479]])
```

- ▶ `requires_grad=True` asks PyTorch to record every operation on that tensor. `backward()` replays the record from the loss back to each dial and stores the slope in `.grad`.
- ▶ The numbers are the hand numbers of the previous frame. From now on you write the forward pass; the backward pass is free.

Neural Networks: **PyTorch** Basics

Definition

A **tensor** is PyTorch's array: the same shapes, slicing and @ as NumPy (Day 05), plus (1) **autograd**, the recorded backward pass, and (2) one line to run on a **GPU**.

NumPy	PyTorch
np.array	torch.tensor
a.shape, a @ b	the same
no slopes	loss.backward()
CPU only	.to("cuda")

```
import torch
X = torch.tensor([[4.7, 1.4], [1.4,
                        0.2]])
X.shape           # torch.Size([2, 2])
X.dtype           # torch.float32
W = torch.ones(2, 3)
(X @ W).shape     # torch.Size([2, 3])
X[:, 0]           # tensor([4.7, 1.4])
X.numpy()         # back to NumPy
```

- ▶ Features are float32; class labels are int64 (torch.tensor([0, 2, 1])).
- ▶ torch.tensor(df.values, dtype=torch.float32) turns a pandas table into a tensor.

```
img = torch.rand(28, 28)           # one 28 x 28 image
img.reshape(-1).shape              # torch.Size([784]): flattened
batch = torch.rand(64, 28, 28)    # 64 images at once
batch.reshape(64, -1).shape       # torch.Size([64, 784]): a table
batch.mean(dim=0).shape           # torch.Size([28, 28]): mean image
torch.from_numpy(np_array)        # shares memory with NumPy
torch.manual_seed(0)              # the same random dials every run
```

- ▶ A network reads a **table**: n rows, one row per sample. Images are flattened into rows first; Week 4 shows the better way (convolutions).
- ▶ The errors you will meet: shapes that do not match (mat1 and mat2 shapes cannot be multiplied) and a float tensor where an int64 label was expected. Both are read from the message.

```
import torch.nn as nn
layer = nn.Linear(64, 32)                # W: 32 x 64, b: 32, random
layer.weight.shape                       # torch.Size([32, 64])

model = nn.Sequential(nn.Linear(64, 32), # hidden: 32 new columns
                      nn.ReLU(),
                      nn.Linear(32, 10)) # output: 10 scores (logits)
logits = model(X)                       # X: n x 64 -> n x 10
sum(p.numel() for p in model.parameters()) # 2410
```

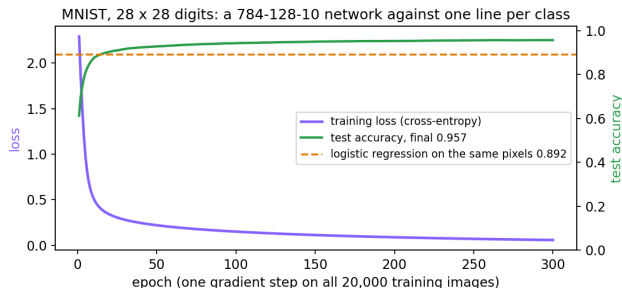
- ▶ `nn.Linear(in, out)` is one layer of out neurons reading in inputs: it computes $\mathbf{XW}^T + \mathbf{b}$, exactly the matrix frame.
- ▶ `nn.Sequential` runs the pieces in order; `model(X)` is the whole forward pass. Every dial already has `requires_grad=True`.
- ▶ No softmax at the end: the loss function applies it (next frame). The outputs are logits.

```
loss_fn = nn.CrossEntropyLoss()           # softmax + log loss
opt = torch.optim.SGD(model.parameters(), lr=0.1) # the update rule

for epoch in range(300):
    logits = model(X_train)                # 1. predict
    loss = loss_fn(logits, y_train)         # 2. loss (int64 labels)
    opt.zero_grad()                        # clear the old slopes
    loss.backward()                        # 3. gradient, by backprop
    opt.step()                             # 4. update: w -= lr * grad
```

- ▶ Day 08's loop: `backward()` replaces the hand-written gradient lines, `step()` the update line. `zero_grad()` matters: PyTorch *adds* new slopes to `.grad`.
- ▶ An **epoch** is one pass over the training rows, here all at once; Day 16 cuts them into mini-batches with a `DataLoader` and names the loss functions one by one.

The Result on MNIST: the Loss Falls, the Accuracy Climbs



Source: MNIST (LeCun et al., 1998) via OpenML `mnist_784`, 20,000 training and 10,000 test images, pixels standardised; the loop of the previous frame with a 784-128-10 network, 300 steps; `scripts/day15.nn.figures.py`.

- ▶ 101,770 dials, 300 steps, and the network passes the line (one logistic regression per digit, 0.892) by 6 points: **0.957**. Nobody wrote a column.
- ▶ The curve is read like Day 09's: still falling at step 300, so more steps would help; Day 16 and 17 show when to stop and how to go faster.

Neural Networks: **Olympiad Practice**

Task

Write `train_neuron(features, labels, initial_weights, initial_bias, learning_rate, epochs)`: a single neuron with a sigmoid activation, trained by gradient descent on the mean squared error. Return the final weights, the final bias and the list of MSE values, one per epoch, all rounded to 4 decimals.

```
train_neuron([[1.0, 2.0], [2.0, 1.0], [-1.0, -2.0]], [1, 0, 0],  
             [0.1, -0.2], 0.0, 0.1, 2)  
# expected: ([0.1036, -0.1425], -0.0167, [0.3033, 0.2942])
```

- ▶ **Question 1:** which three slopes does the chain rule multiply for $\partial \text{MSE} / \partial w_j$, and what is the extra factor compared with Day 08's line?
- ▶ **Question 2:** the MSE is recorded *before* each update. Why do the two numbers above fall, and what would a rising list mean?

Adapted from the Deep-ML open problem bank, problem 25 "Single Neuron with Backpropagation" (github.com/Open-Deep-ML/DML-OpenProblem, contributed by evintunador); the expected output is the bank's own test case.

Recognise: one neuron, squared error, sigmoid: the three-link chain of this deck, averaged over the rows, then the four moves.

```
def train_neuron(features, labels, initial_weights, initial_bias, learning_rate, epochs):
    X, y = np.array(features, float), np.array(labels, float)
    w, b, mse_values = np.array(initial_weights, float), initial_bias, []
    for _ in range(epochs):
        a = 1 / (1 + np.exp(-(X @ w + b))) # 1. predict
        mse_values.append(round(float(np.mean((a - y) ** 2)), 4)) # 2. loss, before the
            step
        delta = 2 * (a - y) * a * (1 - a) / len(y) # 3. the error signal
            per row
        w = w - learning_rate * (X.T @ delta) # 4. update
        b = b - learning_rate * delta.sum()
    return np.round(w, 4).tolist(), round(float(b), 4), mse_values
```

1. $\partial \text{MSE} / \partial w_j = \frac{2}{n} \sum_i (a_i - y_i) a_i (1 - a_i) x_{ij}$: loss slope, sigmoid slope, input; $a(1 - a)$ is the factor Day 08's line did not have.
2. Each step moves against the slope, so the next recorded MSE is smaller: 0.3033, then 0.2942; a rising list means a learning rate too big (Day 08). Check: autograd gives the same slopes, and the output matches the bank's exactly.

Neural Networks: **Limitations & What Comes Next**

- ▶ **On a small table the line or the forest often wins.** Admissions: logistic regression 0.950, forest 0.935 (Day 10). A network needs many rows to invent columns worth having; its home ground is images, sound and text.
- ▶ **Many dials, no formula.** Layers, units, learning rate, number of steps: every one is a choice, and Day 08's formula for the line does not exist here. Day 17 is about choosing them.
- ▶ **Not readable.** 101,770 numbers explain nothing a person can check; the forest's importances and the tree's rules do.
- ▶ **Training fails quietly.** A loss stuck at $\ln 10 = 2.303$ (random guessing over 10 classes) or printing `nan` is a learning-rate or a scaling problem, not a bug in PyTorch.

The common mistake

Feeding raw columns on different scales (pixels 0 to 255 next to flags 0 or 1) and wondering why the loss does not move. Scale first, as for KNN and Ridge; then compare with the majority baseline and with the line before trusting any number.

- ▶ **Tomorrow (Day 16), Loss Functions, Training Loop, Datasets and DataLoaders:** why `CrossEntropyLoss` wants logits, mini-batches instead of the whole table, validation inside the loop, and how to read the loss curve.
- ▶ **Monday (Day 17):** optimizers beyond plain gradient descent (momentum, Adam), learning-rate schedules, BatchNorm, dropout and early stopping: the dials that make training reliable.
- ▶ **Days 18 to 20, self-study:** activation functions in depth, tensors and DataLoaders in depth, then the Olympiad habits for training networks.
- ▶ **Week 4:** networks that read images without flattening them: convolutions.

Neural Networks: **Summary**

- ▶ **A neuron** is a line plus a bend: $a = f(\mathbf{w} \cdot \mathbf{x} + b)$. With the sigmoid it is Day 08's logistic regression; one neuron draws one line.
- ▶ **Activations** are the bend: without one, stacked lines are one line. ReLU inside, sigmoid or softmax at the output according to the task.
- ▶ **A network** stacks layers of neurons; the hidden layer's outputs are columns the network invented. The forward pass is matrix multiply, bend, matrix multiply.
- ▶ **The chain rule** multiplies the slopes of a chain; **backpropagation** applies it layer by layer from the loss back to every dial, and `loss.backward()` does it for you.
- ▶ **PyTorch**: tensors, `nn.Linear`, `nn.Sequential`, and the four-move loop with `zero_grad`, `backward`, `step`.
- ▶ **When it wins**: 0.957 against 0.892 on 28×28 digits; on the admissions table the line stayed ahead. Baseline first, always.

Neural Networks: **References & Resources**

- [1] Rosenblatt, F. (1958). The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological Review* 65(6).
- [2] Minsky, M. and Papert, S. (1969). *Perceptrons*. MIT Press. The XOR argument of the neuron section.
- [3] Rumelhart, D., Hinton, G. and Williams, R. (1986). Learning representations by back-propagating errors. *Nature* 323. Backpropagation as used in the chain-rule section.
- [4] Paszke, A. et al. (2019). PyTorch: an imperative style, high-performance deep learning library. *NeurIPS* 32. <https://pytorch.org/docs>
- [5] LeCun, Y., Cortes, C. and Burges, C. *The MNIST database of handwritten digits*; loaded through OpenML (mnist_784) with `sklearn.datasets.fetch_openml`. Used for the motivation, hidden-unit and loss-curve figures.
- [6] Alpaydin, E. and Kaynak, C. (1998). *Optical Recognition of Handwritten Digits*, UCI repository; the 8×8 version shipped as `sklearn.datasets.load_digits`. Used for the hand-made-columns comparison and the labs.
- [7] Bin Talib, T. *Saudi Arabia Used Cars Dataset*, Kaggle `turkibintalib/saudi-arabia-used-cars-dataset`. The 80 Tahoe listings of Day 09, reused for the line-against-network figure.

- [8] Deep-ML open problem bank, problem 25 *Single Neuron with Backpropagation* (contributed by evintunador). Adapted for the practice task. <https://github.com/Open-Deep-ML/DML-OpenProblem>
- [9] Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, 3rd ed., chapter 10: from biological to artificial neurons, MLPs, and the first training loops.
- [10] All plots: `slides-builder/scripts/day15_nn_figures.py` (matplotlib, PyTorch and scikit-learn, real data only; it also prints every number of the worked examples).